

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

***CO4:** Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

***I D K SUMALATHA(192424023)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D K SUMALATHA

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Given Three-Address Code

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

1. Analysis of the sequence

The operations are performed sequentially:

1. $t1 = x + y$
First, the values of x and y are added and the result is stored in $t1$.
2. $t2 = t1 * z$
The value stored in $t1$ is multiplied by z and stored in $t2$.
3. $t3 = t2 / w$
The value of $t2$ is divided by w and stored in $t3$.
4. $t4 = t3 - p$
Finally, p is subtracted from $t3$, and the result is stored in $t4$.

Therefore, the equivalent mathematical expression is:

2. Equivalent Expression Tree

The expression tree is:

```
      (-)
     /  \
    (/)  \
   /  \  p
  (*)  \
 /  \
(+)  \
  z
```

/ \

x y

The root represents subtraction. Its left child represents division, which contains multiplication and addition operations.

3. Role of Temporary Variables

Temporary variables t1, t2, t3, and t4 divide a complex expression into simple three-address instructions.

For example:

t1 = x + y

t2 = t1 * z

t3 = t2 / w

t4 = t3 - p

Advantages include:

- Each instruction has a simple operation.
- Complex expressions become easier for the compiler to process.
- Temporary results can be stored in registers or memory.
- Dependencies between operations are clearly represented.
- Optimization techniques can easily identify redundant computations.
- Code generation becomes simpler.

Conclusion

Three-address code provides a systematic representation of the expression. Temporary variables make the intermediate representation easier to analyze, optimize, and convert into machine instructions. In this particular sequence, there is no repeated subexpression, so no major common-subexpression elimination is possible.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Given Symbol Table

Variable	Data Type	Size
p	char	1 byte
q	double	8 bytes
r	int	4 bytes
s	float	4 bytes

1. Total Memory Required

The total memory is:

Therefore, the **minimum storage required based only on the declared sizes is 17 bytes.**

2. Possible Memory Organization

A simple sequential representation can be shown as:

Variable	Size	Possible Offset
p	1 byte	0
q	8 bytes	1
r	4 bytes	9
s	4 bytes	13

Total:

However, actual memory layout may contain **padding bytes** because of alignment requirements.

3. Effect of Data Types

Different data types require different amounts of memory:

- char generally requires 1 byte.
- int generally requires 4 bytes.
- float generally requires 4 bytes.
- double generally requires 8 bytes.

The compiler stores these variables according to their size and alignment requirements.

For example, an 8-byte double is commonly aligned on an 8-byte boundary. Therefore, padding may be inserted before q if necessary.

One possible aligned layout is:

Variable	Storage	Explanation
p	1 byte	Stored first
Padding	7 bytes	Aligns q
q	8 bytes	8-byte aligned
r	4 bytes	4-byte aligned
s	4 bytes	4-byte aligned

This would require:

The exact layout depends on the compiler, target architecture, ABI, and whether these variables are local variables, structure members, or independently allocated objects.

Conclusion

The **raw data size is 17 bytes**, while an actual aligned layout can require additional padding. Data types affect both memory consumption and alignment, and the compiler uses the symbol table to maintain information such as name, type, size, and storage location.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Given Intermediate Code

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

1. Step-by-Step Execution

Step 1

```
t1 = m * n
```

The compiler calculates $m \times n$ and stores the result in t1.

Step 2

```
t2 = p + q
```

The compiler calculates $p + q$ and stores it in $t2$.

Step 3

$t3 = m * n$

The same expression $m * n$ is calculated again.

This is redundant because:

$t1 = m * n$

has already calculated the same value.

Step 4

$t4 = t2 - t3$

The value of $t3$ is subtracted from $t2$.

2. Common Subexpression Elimination

The expression:

$m * n$

occurs twice:

$t1 = m * n$

$t3 = m * n$

Therefore, the second calculation can be eliminated.

We can replace:

$t3 = m * n$

with:

$t3 = t1$

Optimized Code

$t1 = m * n$

$t2 = p + q$

$t3 = t1$

$t4 = t2 - t3$

The compiler can also eliminate $t3$ completely because it is only used once:

$t1 = m * n$

$t2 = p + q$

$t4 = t2 - t1$

3. Efficiency Improvement

Original:

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Optimized:

$t1 = m * n$

$t2 = p + q$

$t4 = t2 - t1$

The multiplication $m * n$ is performed **only once instead of twice**.

Conclusion

Common subexpression elimination reduces unnecessary computation. This decreases the number of instructions executed and can improve execution time and resource utilization.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

$\text{if } (x > y \parallel p < q)$

Intermediate code:

$\text{if } x > y \text{ goto L1}$

$\text{if } p < q \text{ goto L1}$

goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Boolean Expression

$\text{if } (x > y \parallel p < q)$

Given Intermediate Code

$\text{if } x > y \text{ goto L1}$

$\text{if } p < q \text{ goto L1}$

goto L2

L1: ...

1. Interpretation of Control Flow

The Boolean expression uses the logical OR operator:

$x > y \parallel p < q$

For OR, the complete condition is true if **at least one** of the two conditions is true.

The intermediate code first evaluates:

if $x > y$ goto L1

If $x > y$ is true, execution immediately jumps to L1.

Therefore, the second condition:

$p < q$

does not need to be evaluated.

If $x > y$ is false, execution continues to:

if $p < q$ goto L1

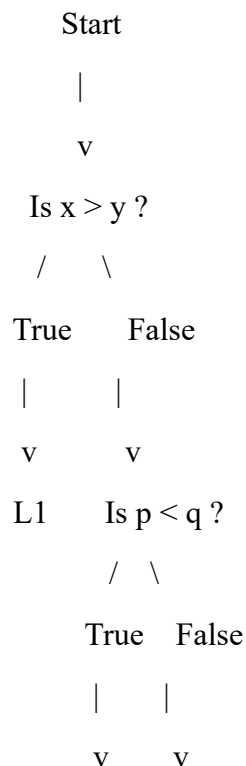
If $p < q$ is true, execution jumps to L1.

If both conditions are false, execution reaches:

goto L2

Thus, L2 represents the false branch.

Control Flow Diagram



L1 L2

2. Short-Circuit Evaluation

Short-circuit evaluation means the compiler does not evaluate unnecessary conditions.

For:

$A \parallel B$

if A is true, B does not need to be checked.

Here:

$x > y \parallel p < q$

If:

$x > y$

is true, the result of the entire expression is already true.

Therefore:

$p < q$

is skipped.

Conclusion

The intermediate representation efficiently implements OR using conditional jumps. Short-circuit evaluation reduces unnecessary condition checking and can improve execution efficiency, particularly when evaluating the second condition is expensive.

Q5. Backpatching and Flow Control

Given:

Instruction	Target
200: if $x == y$ goto _	
201: goto _	

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

Given Instructions

Instruction	Target
-------------	--------

200: if x==y goto _	Unknown
201: goto _	Unknown

Backpatch Lists

truelist = {200}

falselist = {201}

1. Meaning of Backpatching

Backpatching is a technique used during intermediate code generation to temporarily leave jump targets unknown and fill them in later when the destination address becomes known.

The underscore _ represents an unknown target.

Initially:

200: if x == y goto _

201: goto _

The compiler knows that instruction 200 is the true branch and instruction 201 is the false branch, but the final target addresses have not yet been determined.

2. True List

Given:

truelist = {200}

Instruction 200 is associated with the true outcome.

When the destination of the true branch becomes known, instruction 200 is updated.

For example, if the true block begins at instruction 205:

200: if x == y goto 205

3. False List

Given:

falselist = {201}

Instruction 201 represents the false branch.

Suppose the false block begins at instruction 210.

Then:

201: goto 210

The final code becomes:

200: if x == y goto 205

201: goto 210

Before Backpatching

200: if $x == y$ goto _

201: goto _

After Backpatching

200: if $x == y$ goto 205

201: goto 210

4. Importance of Backpatching

Backpatching is important because:

1. Jump destinations may not be known when the instruction is initially generated.
2. It supports Boolean expressions and conditional statements.
3. It helps generate intermediate code for if, if-else, while, and other control-flow statements.
4. It avoids requiring the compiler to know all instruction addresses in advance.
5. It simplifies control-flow translation.

Conclusion

Backpatching connects incomplete jump instructions with their correct target addresses after those targets become known. The truelist and falselist maintain the locations of incomplete jumps, allowing the compiler to construct correct control flow during intermediate code generation.